

Introductie

INFO

Lesinhoud

Doelstellingen

De student(e) kan:

- Een single-page frontend applicatie ontwikkelen met Vue.js.
- Zelfstandig de bovenstaande technologieën combineren tot een werkende webapplicatie.

▼ Inhoud

- [Officiële Vue.js-tutorial](#)

Officiële Vue.js-tutorial

Leer Vue kennen via de [officiële Vue.js-tutorial](#). Je volgt de introductie volledig in de webbrowser via deze tutorial. In latere hoofdstukken gebruik je Vue.js via NPM in een lokaal project.

Dit is een eerste kennismaking met Vue.js, waarbij de onderstaande concepten aan bod komen:

- Declarative Rendering (`{{ }}`)

Inzichtsvraag

Wat doet de dubbele accoladesyntax (`{{ }}`) in Vue.js?

▼ Antwoord

De dubbele accoladesyntax (`{{ }}`) in Vue.js gebruik je voor declaratieve rendering. Hiermee kan je dynamische data uit de Vue-component weergeven in de HTML-template. Wanneer je data verandert, werkt Vue de weergegeven inhoud automatisch bij om de nieuwe waarde weer te geven.

- Reactivity (`ref` , `reactive`)

Inzichtsvraag

Waarom wordt er in Vue gebruik gemaakt van een `ref` of `reactive` om data reactief te maken, in plaats van gewoon een variabele te gebruiken?

▼ Antwoord

In Vue.js worden `ref` en `reactive` gebruikt om data reactief te maken, zodat wanneer deze data verandert, de bijbehorende UI automatisch wordt bijgewerkt. Gewone variabelen zijn niet reactief, waardoor wijzigingen van de data niet automatisch de UI zal bijwerken.

- Attribute Binding (`v-bind` , `:`)

Inzichtsvraag

Wat is het verschil tussen:

```
<script setup>
import { ref } from "vue";
const inputValue = ref("Hello World");
```

vue

```
</script>

<template>
  <input :value="inputValue" />
</template>
```

en

```
<script setup>
import { ref } from "vue";
const inputValue = ref("Hello World");
</script>

<template>
  <input value="inputValue" />
</template>
```

vue

▼ Antwoord

In het eerste voorbeeld ken je de waarde van de reactieve variabele `inputValue` toe aan het `value`-attribuut van het `<input>`-element via Vue's data binding syntax (`:` of `v-bind:`). Hierdoor werkt Vue het invoerveld automatisch bij wanneer `inputValue` verandert. De waarde van `value` is dus `"Hello World"` .

In het tweede voorbeeld ken je de string `"inputValue"` letterlijk toe aan het `value`-attribuut van het `<input>`-element. Hierdoor zal het invoerveld altijd de tekst `"inputValue"` weergeven, ongeacht de waarde van de variabele `inputValue` . Er is geen data binding in dit geval. De waarde van `value` is dus `"inputValue"` .

- Event Listeners (`v-on` , `@`)

Inzichtsvraag

Wat is het verschil tussen:

vue

```
<script setup>
function showAlert() {
  alert("Button clicked!");
}
</script>

<template>
  <button @click="showAlert">Click me</button>
</template>
```

en

vue

```
<script setup>
function showAlert() {
  alert("Button clicked!");
}
</script>

<template>
  <button onclick="showAlert">Click me</button>
</template>
```

▼ Antwoord

In het eerste voorbeeld koppel je de `showAlert` -functie aan het `click` -event van de knop via Vue's event binding syntax (`@` of `v-on:`). Hierdoor roept Vue de functie aan wanneer je op de knop klikt, en verschijnt de alert.

In het tweede voorbeeld probeer je de `showAlert` -functie te koppelen aan het `onclick` -attribuut van de knop, dit is de Vanilla JavaScript-manier. Dit werkt niet zoals verwacht in Vue.js, omdat Vue's reactiviteitssysteem en event handling niet op deze manier functioneren. De alert verschijnt niet wanneer je op de knop klikt.

- Form Bindings (`:` , `@` , `v-model`)

Inzichtsvraag

Wat is het verschil tussen:

```
<script setup>
import { ref } from "vue";
const inputValue = ref("");
</script>
<template>
  <input :value="inputValue" @input="inputValue =
    $event.target.value" />
</template>
```

vue

en

```
<script setup>
import { ref } from "vue";
const inputValue = ref("");
</script>
<template>
  <input v-model="inputValue" />
</template>
```

vue

▼ Antwoord

Er is functioneel geen verschil tussen de twee voorbeelden, in beide gevallen koppel je de waarde van `inputValue` aan het `<input>` - element en werk je `inputValue` bij wanneer je iets typt in het invoerveld.

`v-model` is een eenvoudige manier om `two-way binding` te voorzien.

- Conditional Rendering (`v-if` , `v-else`)

Inzichtsvraag

Bekijk onderstaande code.

```
<script setup>
import { ref } from "vue";
const isVisible = ref(true);
</script>

<template>
  <div v-if="isVisible">This is visible</div>
  <div v-else>This is hidden</div>

  <div v-show="isVisible">This is visible</div>
  <div v-show="!isVisible">This is hidden</div>
</template>
```

Wat is het verschil tussen het gebruik van `v-if` / `v-else` en `v-show` voor conditionele rendering in Vue.js?

▼ Antwoord

Het belangrijkste verschil tussen `v-if` / `v-else` en `v-show` is hoe deze omgaan met de DOM.

- `v-if` / `v-else` : Deze directives voegen elementen toe aan de DOM of verwijderen deze volledig op basis van de voorwaarde.
- `v-show` : Deze directive verandert alleen de CSS `display` - eigenschap van het element om het zichtbaar of onzichtbaar te maken, maar het element blijft in de DOM aanwezig.

Dit is te vergelijken met `display: none;` in CSS voor `v-show` . Terwijl `v-if` / `v-else` het element helemaal niet rendert als de voorwaarde niet waar is.

- List Rendering (`v-for` , `:key`)

Inzichtsvraag

Waarom is het belangrijk om een unieke `:key` toe te voegen bij het gebruik van `v-for` in Vue.js?

▼ Antwoord

Het is belangrijk om een unieke `:key` toe te voegen bij het gebruik van `v-for` omdat dit Vue helpt om efficiënt te bepalen welke items in de lijst je hebt gewijzigd, toegevoegd of verwijderd. Dit verbetert de prestaties van de rendering en zorgt ervoor dat de componenten correct worden bijgewerkt zonder onnodig opnieuw te renderen.

- Computed Property (`computed`)

Inzichtsvraag

Wat is het voordeel van het gebruik van een computed property in plaats van een gewone methode in Vue.js?

▼ Antwoord

Voert Vue een `computed` property automatisch opnieuw uit wanneer één van de gebruikte reactieve variabelen die nodig zijn om de `computed value` te berekenen, verandert. Vue cacheet `computed` properties automatisch en berekent deze alleen opnieuw als één van de afhankelijkheden verandert.

- Lifecycle en Template Refs (`onMounted` , `ref` in template)

Inzichtsvraag

Wat is het verschil tussen `onMounted` en `onBeforeMount` in de Vue.js lifecycle hooks?

▼ Antwoord

- `onBeforeMount` : Vue roept deze hook aan vlak voordat de component voor het eerst in de DOM wordt toegevoegd. Op dit punt rendert de template nog niet.

- `onMounted` : Vue roept deze hook aan nadat de component in de DOM is toegevoegd en de template heeft gerenderd. Op dit punt zijn alle DOM-elementen beschikbaar voor manipulatie.

- Watchers (`watch`)

Inzichtsvraag

Wanneer wordt een `watch` gebruikt in plaats van een `computed` property in Vue.js?

▼ Antwoord

Gebruik je een `watch` wanneer je moet reageren op veranderingen van een specifieke reactieve variabele, waarbij je side effects uitvoert, zoals het aanroepen van een API of het bijwerken van andere variabelen. `computed` properties zijn bedoeld voor het afleiden van waarden zonder side effects.

- Components

- Props
- Emits
- Slots

Inzichtsvraag

Wat is het verschil tussen props en slots in Vue.js componenten?

▼ Antwoord

- Props: Hiermee geef je data door van een `parent` -component naar een `child` -component.
- Slots: Hiermee geef je data **of content** door van een `parent` -component naar een `child` -component, waardoor de `parent`

bepaalt wat je binnen de `child` weergeeft.

Slots bieden meer flexibiliteit omdat ze niet beperkt zijn tot alleen gegevens, maar ook HTML-structuren kunnen bevatten.

Na het doorlopen van deze tutorial beheers je de basisconcepten van Vue.js. In latere hoofdstukken breid je deze kennis uit en pas je die toe in een lokaal project. Je bespreekt dan ook de projectstructuur van Vue.

Previous page
Oefeningen

Next page
Theorie